MODEL VALIDATION · ENSEMBLE

Model Validation and Ensembles

How to measure model performance fairly, raise it with tuning, and combine models

Cross-Validation, Hyperparameter Tuning and Ensemble Learning

How to measure performance on data the model has not seen, how to find the hyperparameters that raise it, and how to combine models for better predictions than any single model, in that order.

Validate to measure, tune to improve, ensemble to combine

Cross-validation scores guide tuning, and the tuned models are combined into an ensemble

1

Cross-Validation

Review the ways to split data and stratified K-fold, and validate time series without information leakage

2

Hyperparameter Tuning

Find hyperparameters through grid search, random search, then Bayesian optimization

3

Ensemble

Review the structure of bagging and boosting, and practice voting ensembles and average blending

4. Stacking — train a meta-model that takes predictions as input, and use CV to prevent information leakage

01

PART 1 · Cross-Validation

Cross-Validation

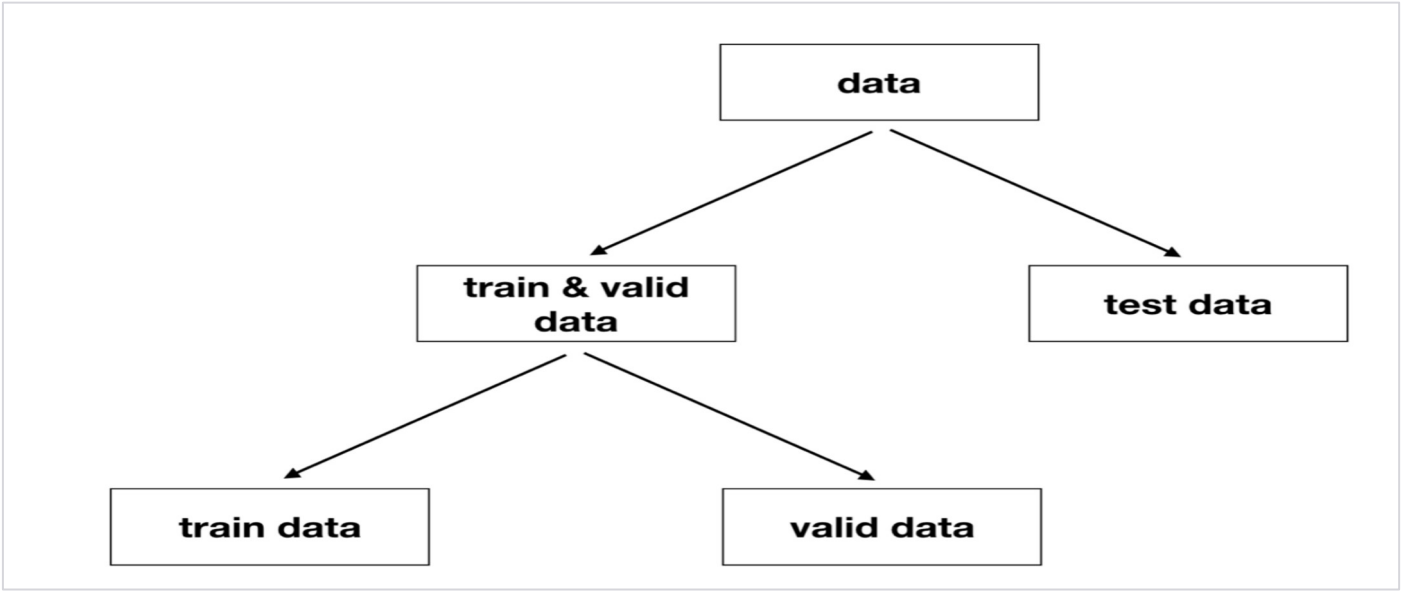
How to measure performance fairly on data the model has not seen

Data is split into three roles: training, validation and test

Train & valid data is split in two again; test data is set aside until the end

Roles of the three datasets

- Training data: data used to train the model
- Validation data: data used to validate the trained model. The model has not seen it; it is used to choose hyperparameters
- Test data: data predicted by the model trained with the chosen hyperparameters. The model has not seen this data either

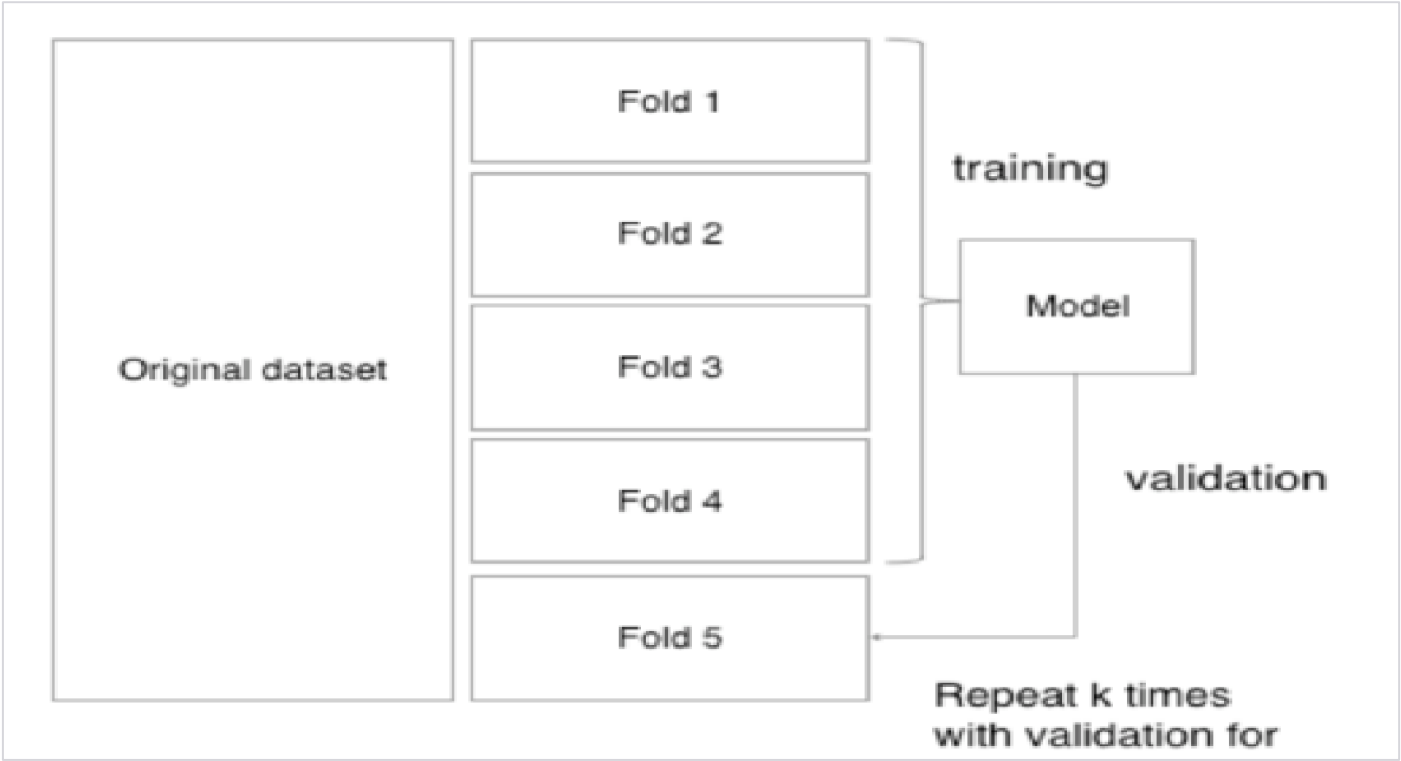


In K-fold CV, every fold serves once as validation data

K-fold: split the data into K parts, train on K−1 and validate on the remaining one

How K-fold works

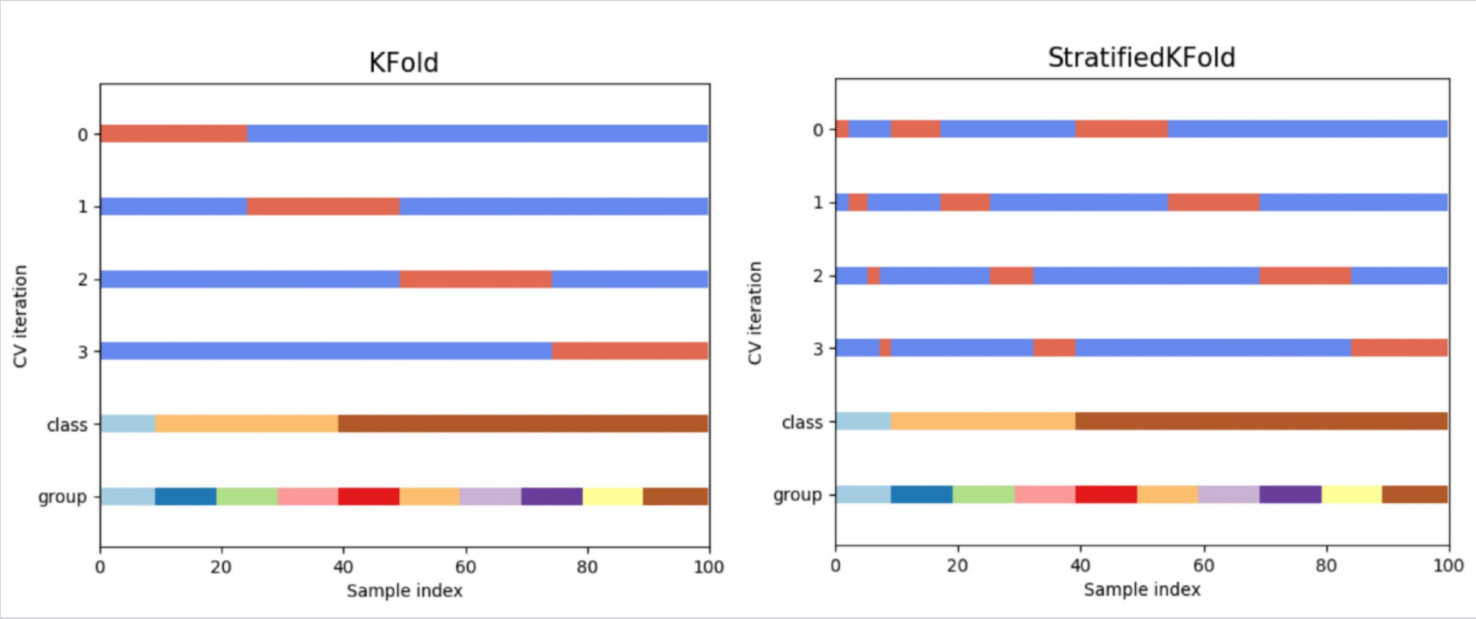
- Split the data into K folds
- Train the model on K−1 folds and validate on the remaining fold
- Rotate the validation fold and repeat K times, so models trained on K different datasets can be validated



Stratified K-fold matches each fold's class ratio to the whole

What stratify means — top: KFold, bottom: StratifiedKFold (red = validation segment, class = class distribution)

- K-fold splits regardless of how the data is sorted or its class ratio. If one class concentrates in a fold, that fold does not represent the training dataset
- Stratified K-fold, after splitting into K parts, keeps each fold's class ratio equal to that of the full training dataset

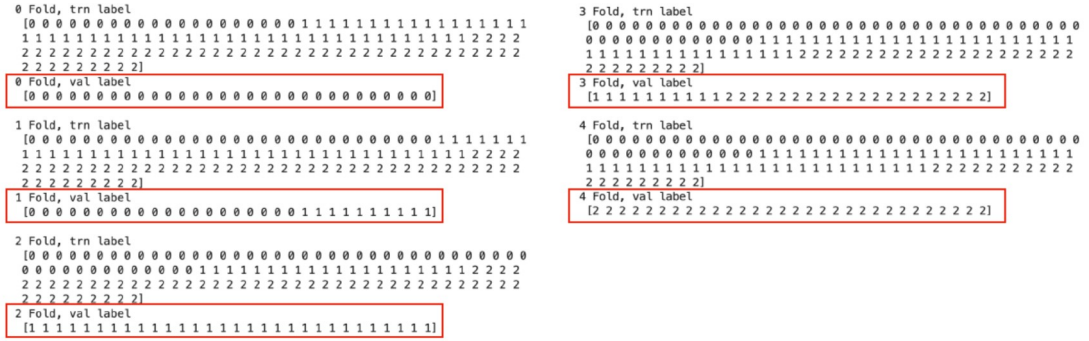


On sorted data, KFold leaves only one class in a validation fold

KFold(n_splits=5) — red boxes mark each fold's validation labels

[Input] Prompt · Claude Code (Code tab)

```
Test KFold on the iris data, keeping the label order
(0→1→2).
1. Split with KFold(n_splits=5), without shuffling.
2. Print the training and validation labels of each
fold.
3. Tabulate the class counts of each validation fold.
4. Use the current sklearn API.
```



What to check

- Do folds 0, 2 and 4 hold a single class?
- Does each fold total 30 samples?
- Is random_state left out when shuffle is off?

Reading the output

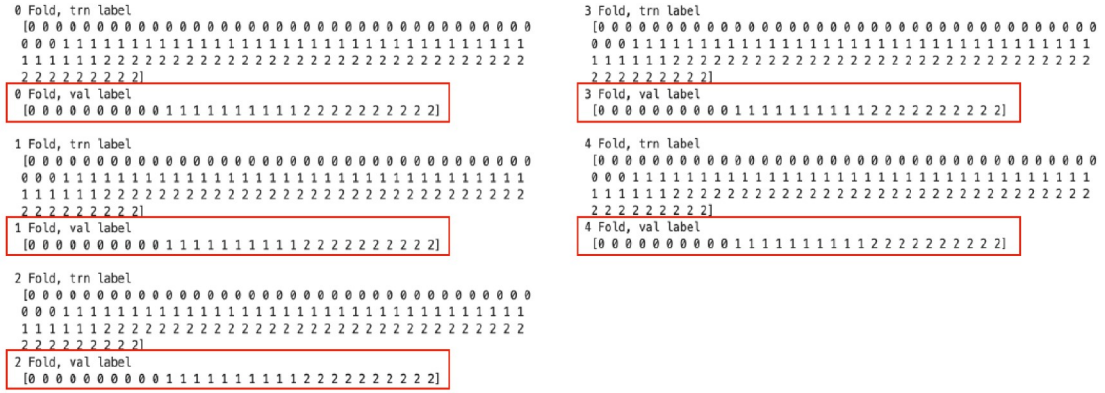
- Labels are sorted in the order 0→1→2
- Validation labels are all 0 in fold 0, all 1 in fold 2 and all 2 in fold 4
- The validation folds do not represent the full distribution

StratifiedKFold spreads all three classes over every fold

StratifiedKFold(n_splits=5) — same data, same loop

[Input] Prompt · Claude Code (Code tab)

```
Test StratifiedKFold(n_splits=5) on the same iris data.
1. With the same loop as before, print the training and
validation labels of each fold.
2. Tabulate the class counts of each validation fold
next to the KFold table.
```



What to check

- Does every validation fold hold 10 per class?
- Are the data and loop the same as for KFold?

Reading the output

- Every fold's validation labels hold 0, 1 and 2 in equal proportions
- Each fold represents the whole dataset
- This becomes the default split for classification problems

The CV score is the mean of per-fold validation accuracy

Validate a random forest (n_estimators=100, max_depth=10) with 5-fold StratifiedKFold

[Input] Prompt · Claude Code (Code tab)

```
Validate a random forest on the iris data with
StratifiedKFold(n_splits=5).
- Model: n_estimators=100, max_depth=10, random_state=2019
- Print each fold's training and validation accuracy as %
with two decimals.
- Finally print the mean validation accuracy as the Cross
Validation Score.
```

What to check

- Is the model refit in every fold?
- Is the mean taken over validation accuracy?
- Is 100% training accuracy not read as performance?

```
0 Fold, train Accuracy : 100.00%, validation Accuracy : 96.67%
1 Fold, train Accuracy : 100.00%, validation Accuracy : 96.67%
2 Fold, train Accuracy : 100.00%, validation Accuracy : 93.33%
3 Fold, train Accuracy : 100.00%, validation Accuracy : 96.67%
4 Fold, train Accuracy : 100.00%, validation Accuracy : 100.00%
Cross Validation Score : 96.67%
```

Results

- Training accuracy is 100% in every fold
- Validation accuracy varies by fold, from 93.33% to 100%
- The mean, 96.67%, is the Cross Validation Score

cross_val_score computes the K-fold CV score in one call

Pass a number to cv and it creates that many folds; pass a fold object and it splits as that object does

[Input] Prompt · Claude Code (Code tab)

```
Compare three settings with cross_val_score on the sorted
iris data.
- Model: RandomForestClassifier(n_estimators=100,
max_depth=10, random_state=2019)
- cv=5, cv=KFold(n_splits=5), cv=StratifiedKFold(n_splits=5)
- Show the mean accuracy as a % table.
```

What to check

- Do cv=5 and StratifiedKFold match?
- Is KFold the lowest?
- Values vary with model settings and versions: are they noted?

Random Forest k-Fold CV Score(Acc) : 96.67%

Random Forest k-Fold CV Score(Acc) : 91.33%

Random Forest Stratify k-Fold CV Score(Acc) : 96.67%

- It returns one score per fold, and their mean evaluates the model. For classifiers it applies StratifiedKFold by default
- On sorted data, the same random forest as the previous slide scores 91.33% with a KFold object and 96.67% with StratifiedKFold. cv=5 (stratified by default) also gives 96.67%, matching the previous slide

Splitting methods differ on shuffling and time order

sklearn splitting methods, illustrated with 10 data points (0–9)

Method	How validation data is formed	Splits
train_test_split	Split once at random (e.g., 80% training)	1
KFold	Cut into 5 folds in order and validate one fold at a time	5
KFold(shuffle=True)	Shuffle first, then cut into 5 folds	5
LeaveOneOut	Hold out one sample at a time for validation	10
LeavePOut(p=2)	Hold out every possible combination of two samples	45
ShuffleSplit	Repeatedly shuffle and draw a fixed number of samples	3
TimeSeriesSplit	Train on the past and validate on the next time point	5

With K folds, every sample is validated once

train_test_split splits once, so luck plays a role; KFold(n_splits=5) validates two samples at a time in order

train_test_split

```
data = list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
train_test_split(data, train_size=.8)
[[8, 7, 4, 10, 1, 3, 5, 2], [6, 9]]
```

Draws 80% training and 20% validation once at random. The score shifts depending on which samples land in validation

KFold

```
kf = KFold(n_splits=5)
for train, validate in kf.split(data):
    print(train, validate)
[2 3 4 5 6 7 8 9] [0 1]
[0 1 4 5 6 7 8 9] [2 3]
[0 1 2 3 6 7 8 9] [4 5]
[0 1 2 3 4 5 8 9] [6 7]
[0 1 2 3 4 5 6 7] [8 9]
```

Validates 0·1, 2·3, ..., 8·9 in turn. If the data is sorted, validation folds skew to one side

Shuffled K-fold avoids sorting; LOO leaves out one sample

KFold(shuffle=True, random_state=42) and LeaveOneOut

KFold with shuffle

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
for train, validate in kf.split(data):
    print(train, validate)
[0 2 3 4 5 6 7 9] [1 8]
[1 2 3 4 6 7 8 9] [0 5]
[0 1 3 4 5 6 8 9] [2 7]
[0 1 2 3 5 6 7 8] [4 9]
[0 1 2 4 5 7 8 9] [3 6]
```

Shuffling before cutting scatters validation folds, as in 1·8 and 0·5. Fix random_state for reproducibility

Leave-one-out CV

```
loo = LeaveOneOut()
for train, validate in loo.split(data):
    print(train, validate)
[1 2 3 4 5 6 7 8 9] [0]
[0 2 3 4 5 6 7 8 9] [1]
...
[0 1 2 3 4 5 6 7 9] [8]
[0 1 2 3 4 5 6 7 8] [9]
```

Trains as many times as there are samples (10). Used for small data, but computationally heavy with many samples

LPO uses all combinations; ShuffleSplit draws at random

LeavePOut(p=2) and ShuffleSplit(n_splits=3, test_size=2)

Leave-p-out CV

```
lpo = LeavePOut(p=2)
for train, validate in lpo.split(data):
    print(train, validate)
[2 3 4 5 6 7 8 9] [0 1]
[1 3 4 5 6 7 8 9] [0 2]
...
[0 1 2 3 4 5 6 8] [7 9]
[0 1 2 3 4 5 6 7] [8 9]
```

Validates every way of leaving out 2 of 10 samples (45 cases). The number of combinations explodes as p grows

Shuffle Split

```
ss = ShuffleSplit(n_splits=3, test_size=2, random_state=42)
for train, validate in ss.split(data):
    print(train, validate)
[5 0 7 2 9 4 3 6] [8 1]
[8 5 3 4 7 9 6 2] [0 1]
[0 6 8 5 3 7 1 4] [9 2]
```

Shuffles each time and draws 2 samples for validation. A sample may be drawn several times or never

In time series, train on the past and validate on the future

TimeSeriesSplit(n_splits=5) — the training span keeps growing; validation is always the very next time point

Why not shuffle?

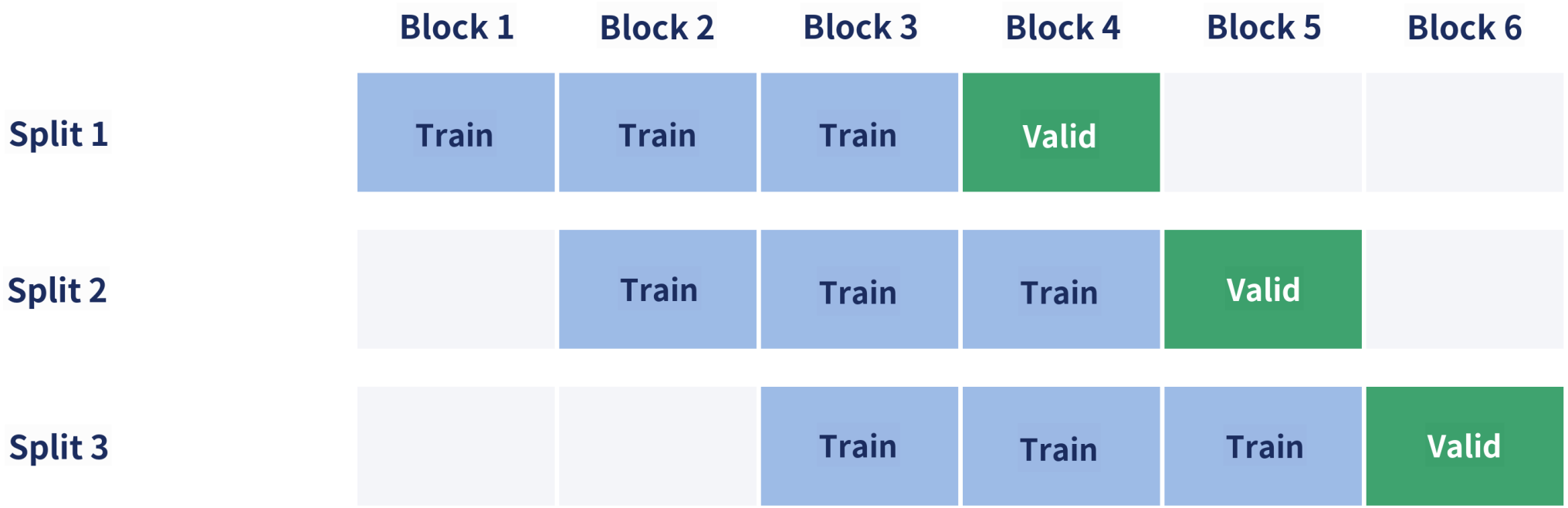
- Shuffling means training on future data to predict the past
- In live use the future cannot be seen, so the validation score is inflated
- So train on 0–4 to validate 5, and on 0–5 to validate 6

```
tscv = TimeSeriesSplit(n_splits=5)
```

```
for train, validate in tscv.split(data):
    print(train, validate)
[0 1 2 3 4] [5]
[0 1 2 3 4 5] [6]
[0 1 2 3 4 5 6] [7]
[0 1 2 3 4 5 6 7] [8]
[0 1 2 3 4 5 6 7 8] [9]
```

Walk-forward slides a fixed training window one step ahead

Walk Forward CV — split the training set into k blocks, train on blocks 1 to p, validate on block p+1, and repeat



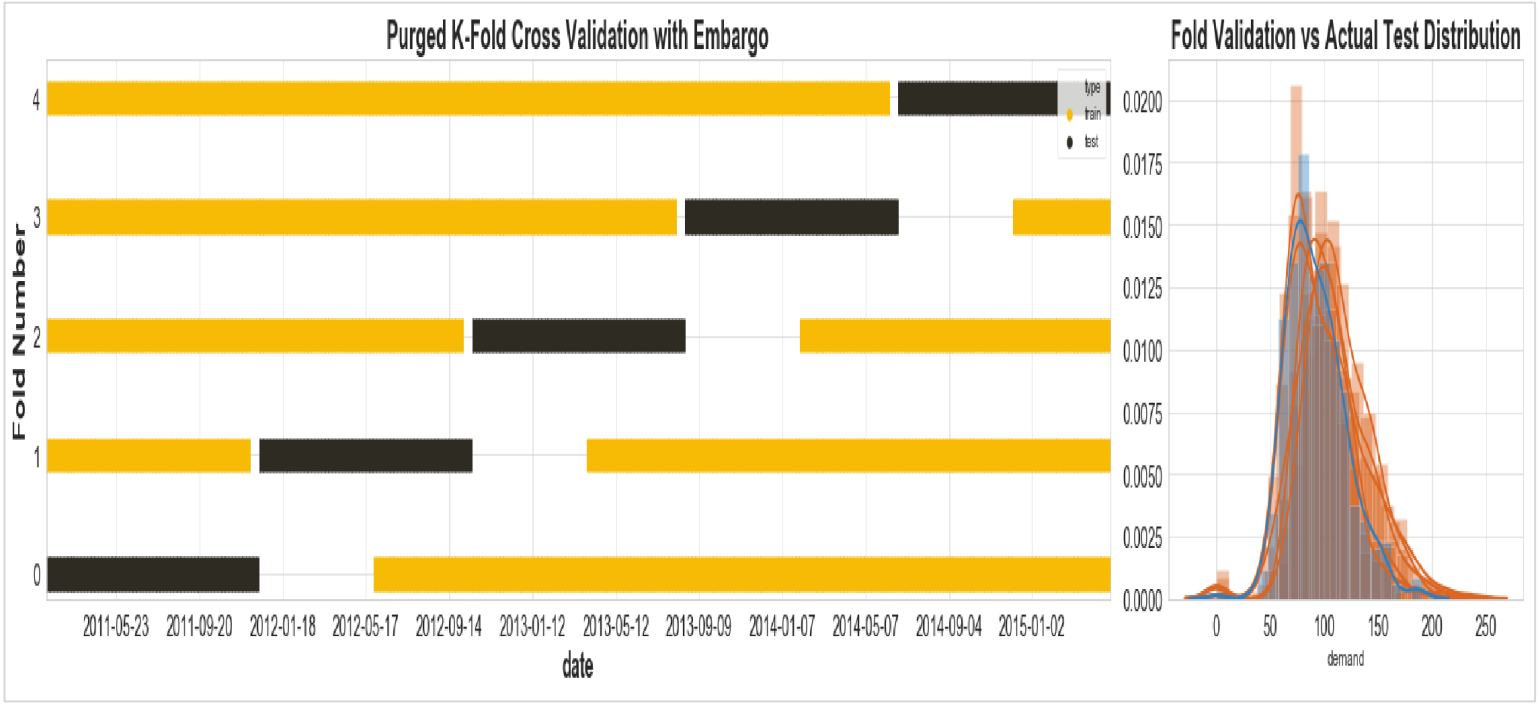
k = 6 blocks, training window p = 3 blocks → sliding one step gives k − p = 3 splits. Validation always follows training

Purged K-fold with embargo validates every time segment

Purged K-Fold CV with Embargo — yellow is training, black is the validation fold, the right panel compares distributions

Reading the figure

- The short gap right after the validation fold (black) is the embargo
- Unlike walk-forward, data after the validation fold is also used for training
- Also check whether each fold's validation distribution resembles the actual test distribution

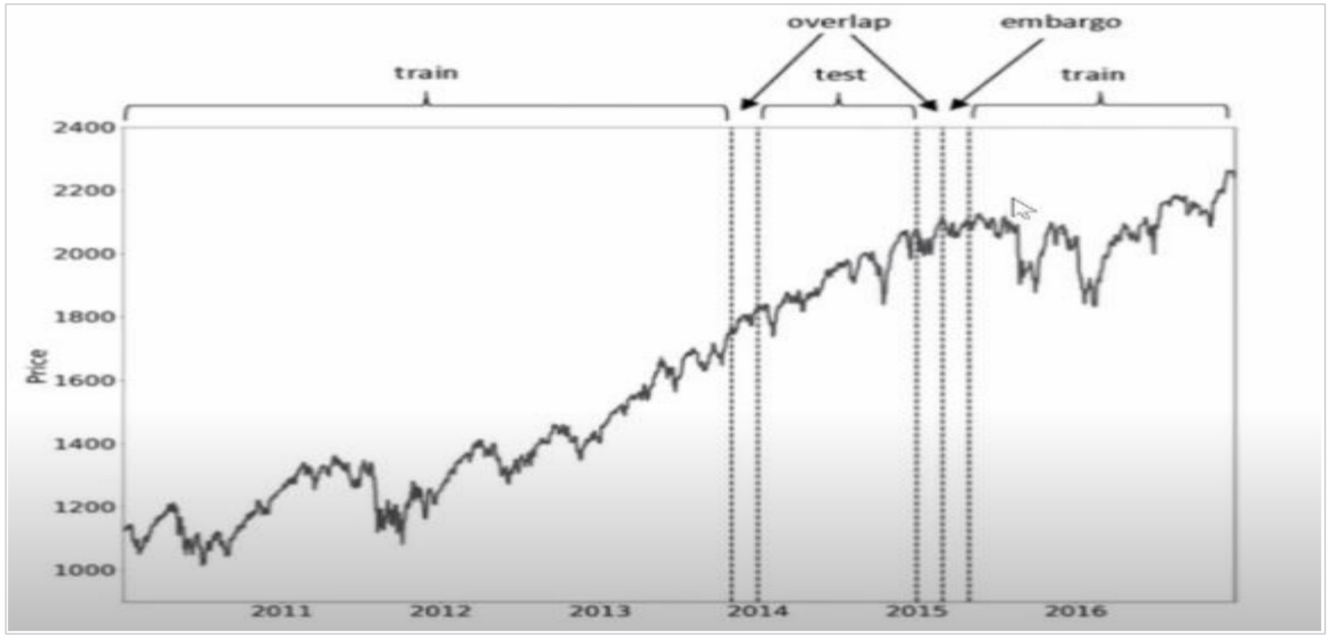


Cross-validation of financial time series has three pitfalls

Overlap and embargo around the test segment — Advances in Financial Machine Learning, Ch. 7

Problems and remedies

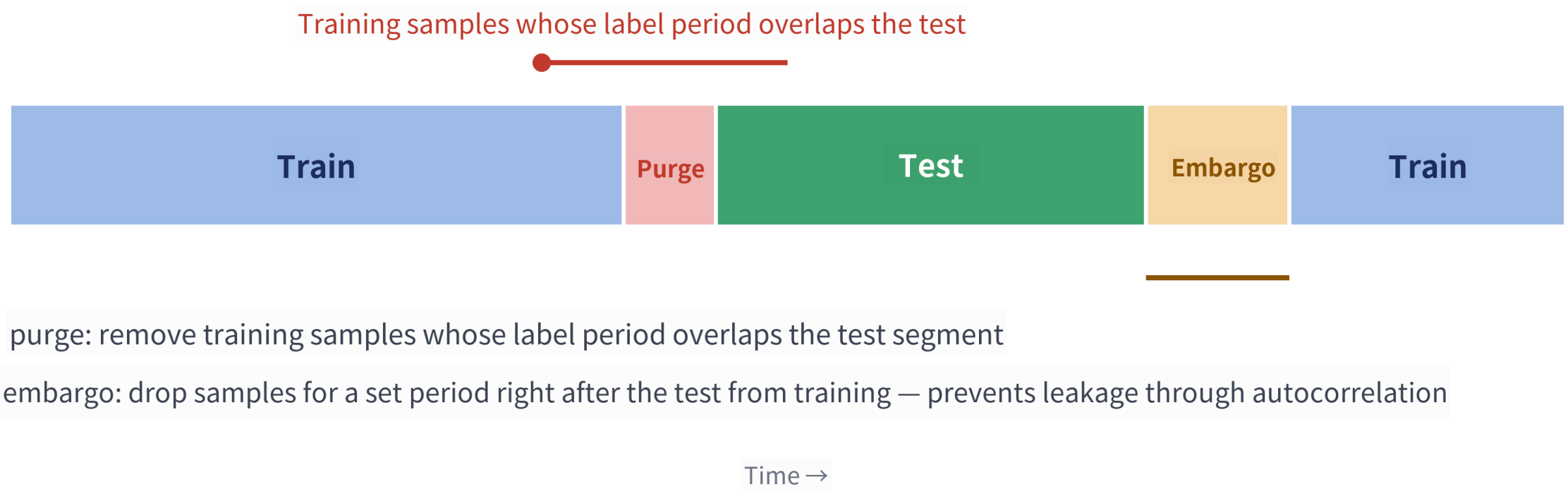
- Training samples whose label spans overlap the test segment carry test information → purge
- Samples right after the test still carry test information through autocorrelation, so that span is also cleared → embargo
- With only one path, the backtest result hinges on a single draw of luck, and the training set is also small → combinatorial path generation



Purging drops overlapping labels; embargo the post-test span

When labels span several days, as with triple barriers, training samples that overlap the test see the answer in advance

Clear both sides of the test segment so future information does not leak into training



purge: remove training samples whose label period overlaps the test segment

embargo: drop samples for a set period right after the test from training — prevents leakage through autocorrelation

Combinatorial purged CV generates multiple backtest paths

Combinatorial Purged CV — uses every combination that picks k of N groups as test (AFML Ch. 12)

$$\underbrace{\varphi[N,k]}_{\text{paths}} = \frac{k}{N}\underbrace{\binom{N}{k}}_{\text{train–test splits}}$$

N : groups, k : test groups

$$N = 6,\ k = 2: \quad \binom{6}{2} = 15 \text{ splits}, \quad \varphi[6,2] = \frac{2}{6} \times 15 = 5 \text{ paths}$$

	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	Paths
G1	x	x	x	x	x											5
G2	x					x	x	x	x							5
G3		x				x				x	x	x				5
G4			x				x			x			x	x		5
G5				x				x			x		x		x	5
G6					x				x			x		x	x	5

FIGURE 12.1 Paths generated for $\varphi[6,2] = 5$

Reading the table

- Rows are the 6 groups, columns the 15 splits
- Each group is tested five times, giving five paths
- View performance as a distribution, not a single number

02

PART 2 · Hyperparameter Tuning

Hyperparameter Tuning

Grid search · random search · Bayesian hyperparameter optimization

How hyperparameters are set drives learning performance

Grid search compares every combination of preset candidate values by cross-validation

Grid Search

- Example: SVC's learning performance depends on C, which sets the degree of soft vs. hard margin; kernel, which sets the kernel function; and gamma, which sets how finely a given kernel looks at the data
- Here GridSearchCV finds the best n_estimators and max_depth for a random forest
- GridSearchCV is in sklearn's model_selection package

Note — scikit-optimize (skopt)

- It is not in sklearn, but the scikit-optimize (skopt) library is available
- Given each parameter's min–max range and the scale of its value distribution, it automates tuning
- skopt's BayesSearchCV performs Bayesian hyperparameter optimization

Grid search compares all 20 combinations to find the best one

5 n_estimators × 4 max_depth = 20 combinations — the best is max_depth 5, n_estimators 50

[Input] Prompt · Claude Code (Code tab)

```
Tune a random forest on the iris data with GridSearchCV.
- n_estimators: 50, 150, 200, 300, 400
- max_depth: 2, 5, 10, 20
- random_state=2019, cv=StratifiedKFold(n_splits=5)
- Print best_score_ (%) and best_params_.
- Save the scores of all 20 combinations as a table.
```

```
GridSearchCV best score : 96.67%,

best_params : {'max_depth': 5, 'n_estimators': 50}
```

What to check

- Were all 20 combinations evaluated?
- If scores tie, which combination was picked?
- Does the best value sit at the edge of the grid?

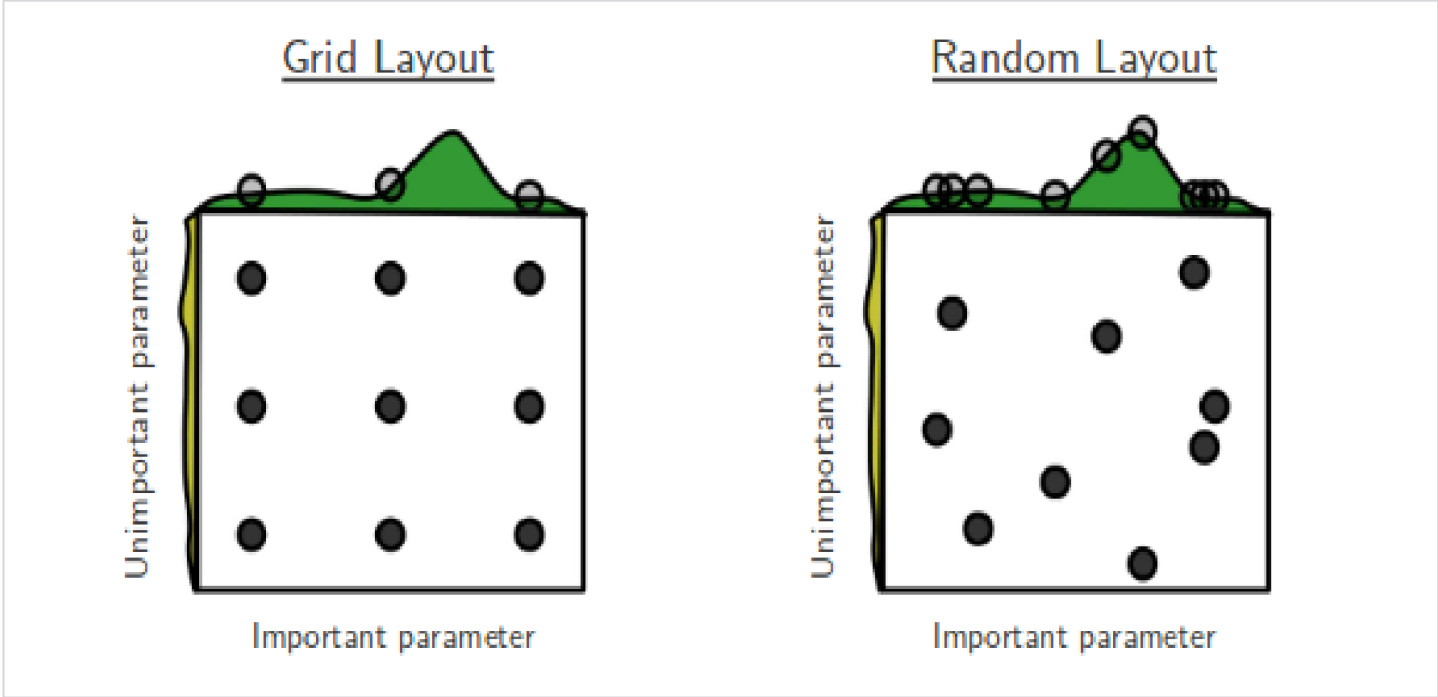
best score 96.67% — the same score as the earlier cross-validation (100 trees, depth 10), from a smaller model

At equal trials, random search covers important values better

Grid Layout vs Random Layout — 9 trials; the top curve is performance over the important parameter

Grid search vs. random search

- Grid's 9 trials see only 3 values of the important parameter
- Random sees 9 different values, so it gets closer to the peak of the performance curve
- Use RandomizedSearchCV in sklearn.model_selection; usage is the same as GridSearchCV

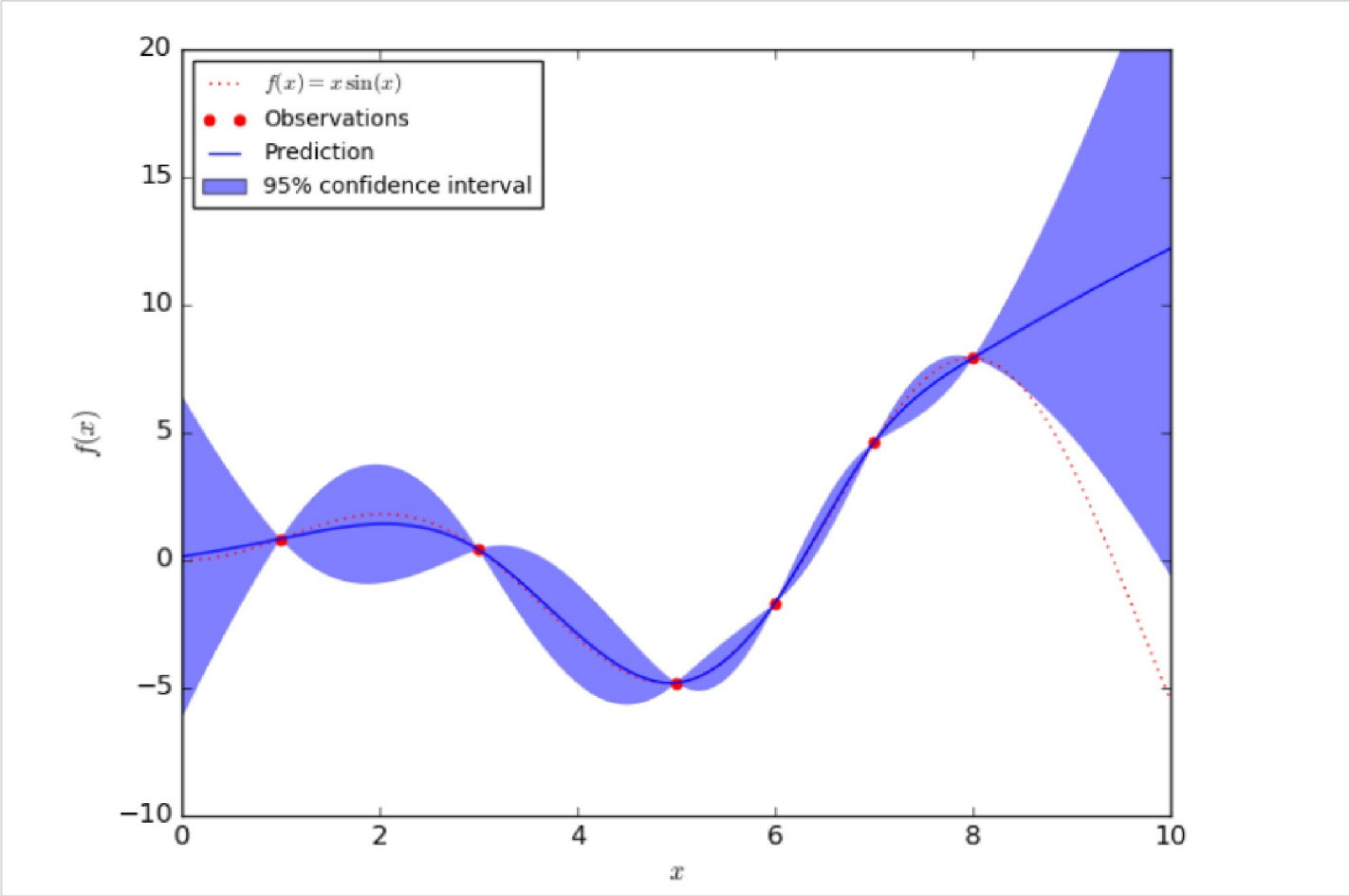


A Gaussian process gives the objective's mean and uncertainty

Surrogate model — red dotted line = true function, red dots = observations, blue band = 95% confidence interval

Reading the figure

- Hyperparameter performance (the objective function) is costly to compute, so only a few points are observed
- The confidence interval is narrow near observed points and widens with distance from them
- This surrogate model selects the next point to observe

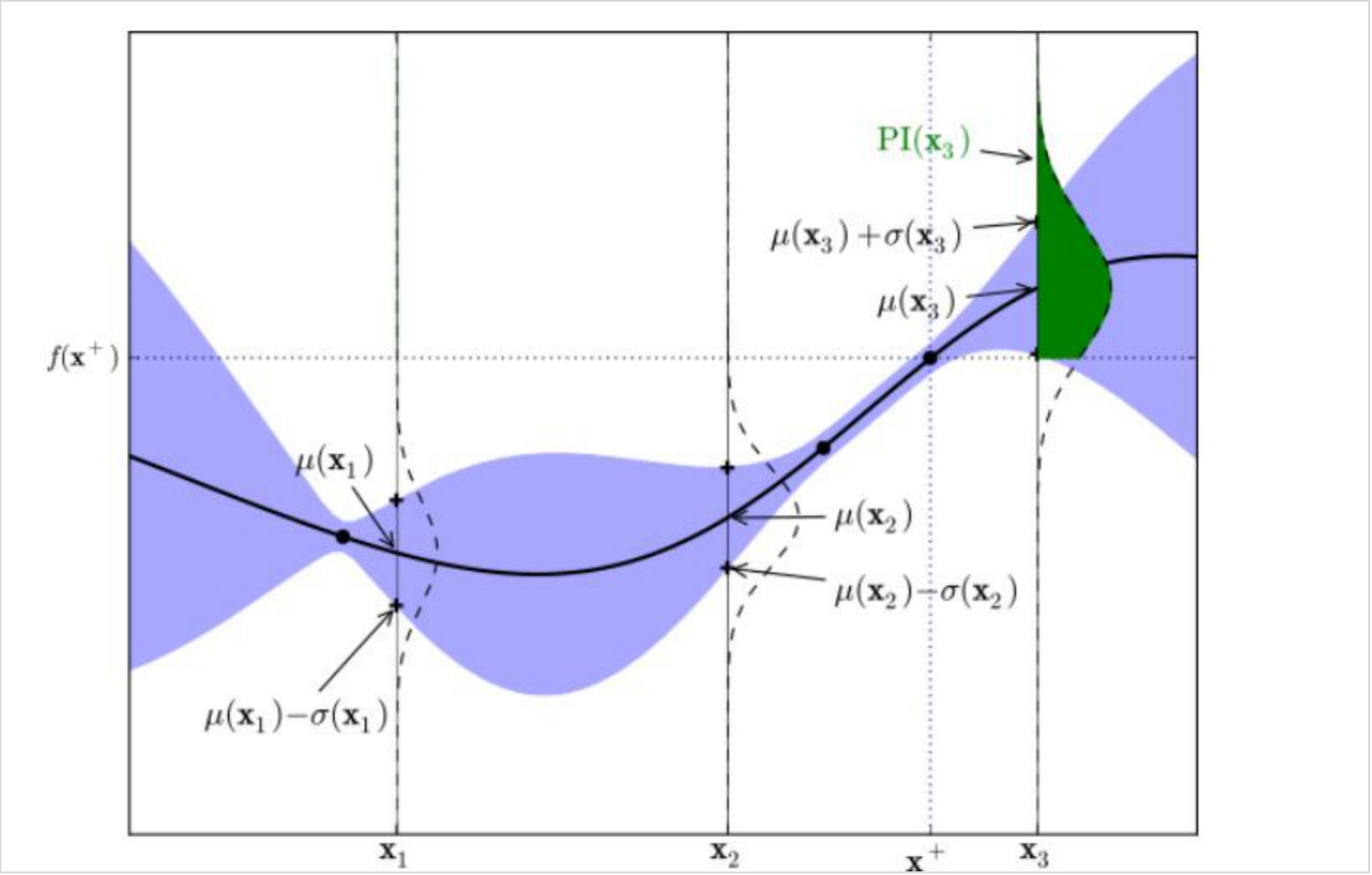


A Gaussian process gives each point a normal prediction

Gaussian process explained — dotted line height is the best value so far; the green area (PI) is the probability of exceeding it

Reading the figure

- Each of the three candidate points shows its mean and ±1 standard deviation
- The rightmost point has a mean above the current best and high uncertainty
- The larger the green area, the more valuable that point is to observe



Acquisition function: next point = largest expected improvement

Acquisition function = function that maximizes expected improvement (EI, Expected Improvement)

$$x^* = \arg\max_{x_i^{\text{new}} \in X^{\text{new}}} \underbrace{\text{EI}(x_i^{\text{new}})}_{\text{expected improvement}},$$

$$\text{EI}(x_i^{\text{new}}) = \mathbb{E}\left[\max\left(0, \underbrace{f(x_i^{\text{new}})}_{\text{new candidate's performance}} - \underbrace{\max_{x_j \in X} f(x_j)}_{\text{best so far}} \right) \right]$$

How to read it

- Only the amount by which a new candidate beats the best performance so far counts as improvement. If it falls short, improvement is 0
- The candidate with the largest expected improvement is observed next (Source: Korea University DMQA Seminar)

Expected improvement = exploitation term + exploration term

The Gaussian process mean and standard deviation give expected improvement in closed form

$$\mathrm{EI}(x) = \begin{cases} \underbrace{\big(\mu(x) - f(x^+) - \xi\big)\Phi(Z)}_{\text{exploitation}} + \underbrace{\sigma(x)\,\phi(Z)}_{\text{exploration}} & \sigma(x) > 0 \\ 0 & \sigma(x) = 0 \end{cases} \qquad Z = \frac{\mu(x) - f(x^+) - \xi}{\sigma(x)}$$

$\mu(x)$: surrogate model's predicted mean

$\sigma(x)$: prediction std. dev.

$f(x^+)$: best value observed so far

ξ : exploration margin (small positive)

Φ : standard normal CDF

ϕ : standard normal density

Exploitation

- Picks where the mean exceeds the current best

Exploration

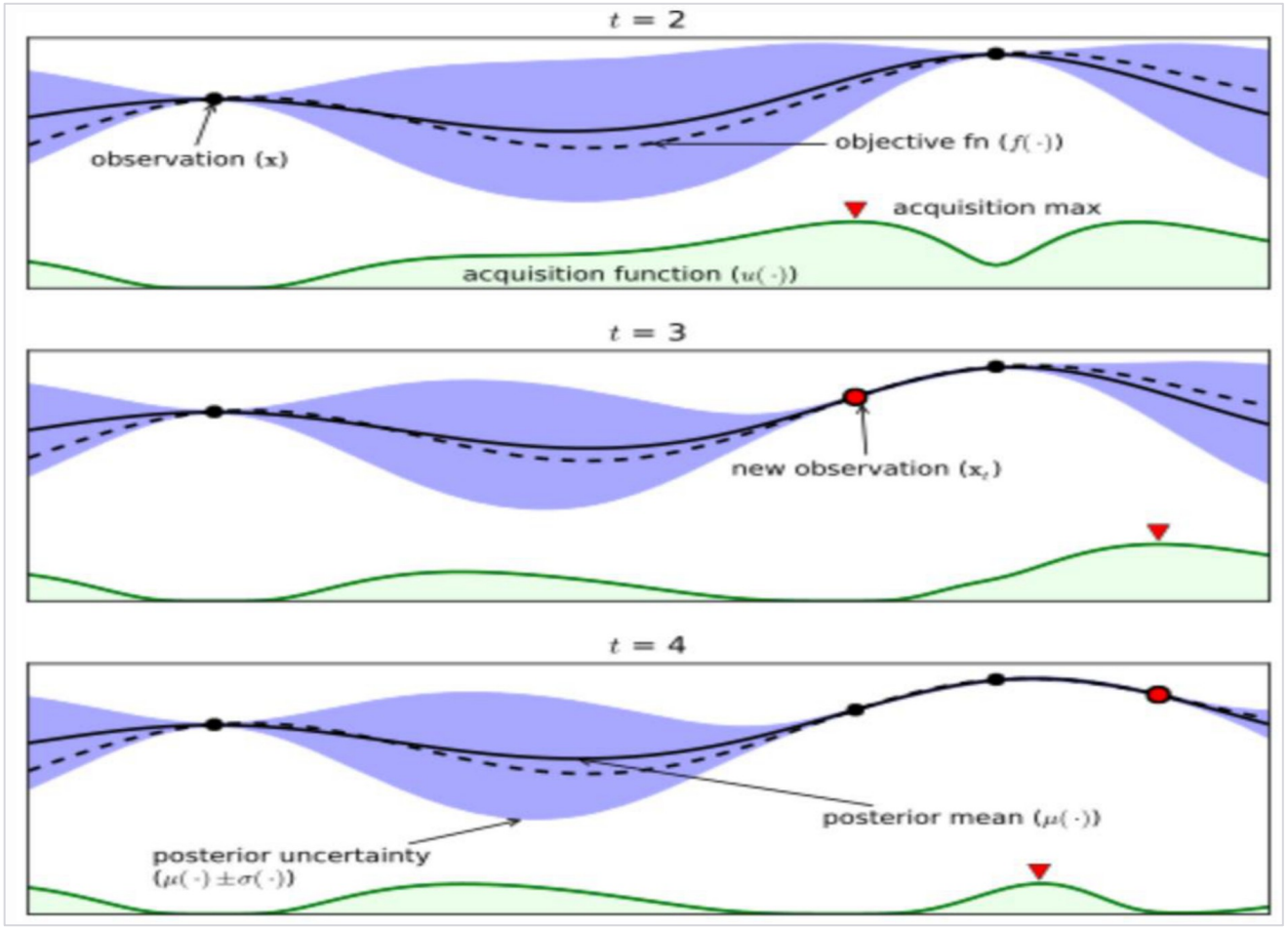
- Picks uncertain places not yet well sampled

With more observations, chosen points approach the optimum

t = 2, 3, 4 — observe the acquisition maximum (red triangle) and re-estimate the surrogate model

Legend

- Black dashed line: true objective function
- Black solid line: mean of the estimated function
- Purple area: standard deviation of the estimated function
- Green area: acquisition function



Bayesian optimization targets the cross-validation score

LightGBM example — BayesianOptimization chooses 6 parameters to maximize the mean AUC from lgb.cv

[Input] Prompt · Claude Code (Code tab)

```
Tune LightGBM with Bayesian optimization (binary classification, X,
y).
- Objective: maximize the mean AUC from lgb.cv
  (nfold=3, stratified, seed=6, n_estimators=100, learning_rate=0.05,
early_stopping_round=100)
- Ranges: num_leaves 24–45, feature_fraction 0.1–0.9, bagging_fraction
0.8–1, max_depth 5–8.99, min_split_gain 0.001–0.1, min_child_weight 5–
50
- init_points=5, n_iter=10, random_state=0
- Print the best parameters and AUC.
```

What to check

- Are num_leaves and max_depth rounded to integers?
- Are feature_fraction and the like kept within 0–1?
- Does the AUC key from lgb.cv match the installed version?
- Does the best value sit at a range edge?

init_points runs random exploration first, then n_iter points chosen by the acquisition function are observed in turn

03

PART 3 · Ensembles

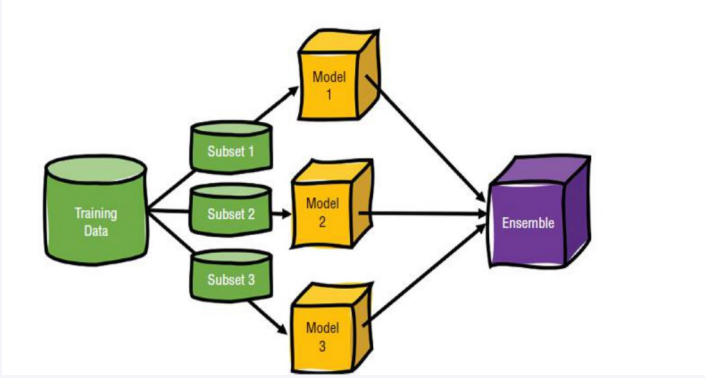
Ensembles

Bagging · boosting · stacking, voting ensembles and average blending

Ensembles fall into three types by how the models are combined

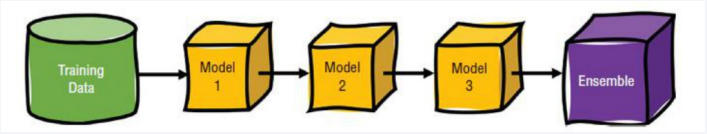
Single-model performance matters, but ensembles that exploit model diversity are the crown jewel of machine learning

Bagging



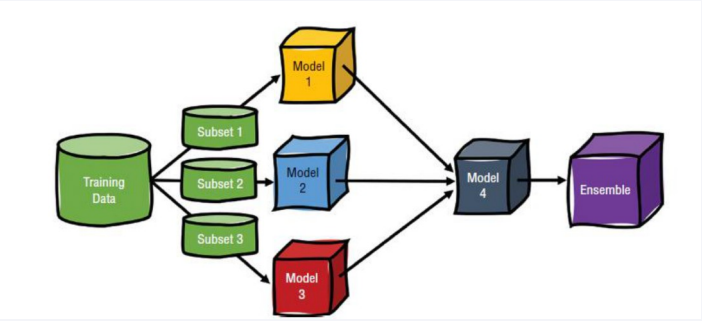
Combines homogeneous models trained independently

Boosting



Applies homogeneous models sequentially

Stacking



Combines independently trained models with a meta-model

The practice covers the three most widely used ensembles

Voting Ensemble · Average Blending · Stacking

1

Voting Ensemble

Each model votes and the majority picks the class. sklearn supports it, so it is easy to use

2

Average Blending

Merges models by averaging predicted values or class probabilities. The most common choice on Kaggle

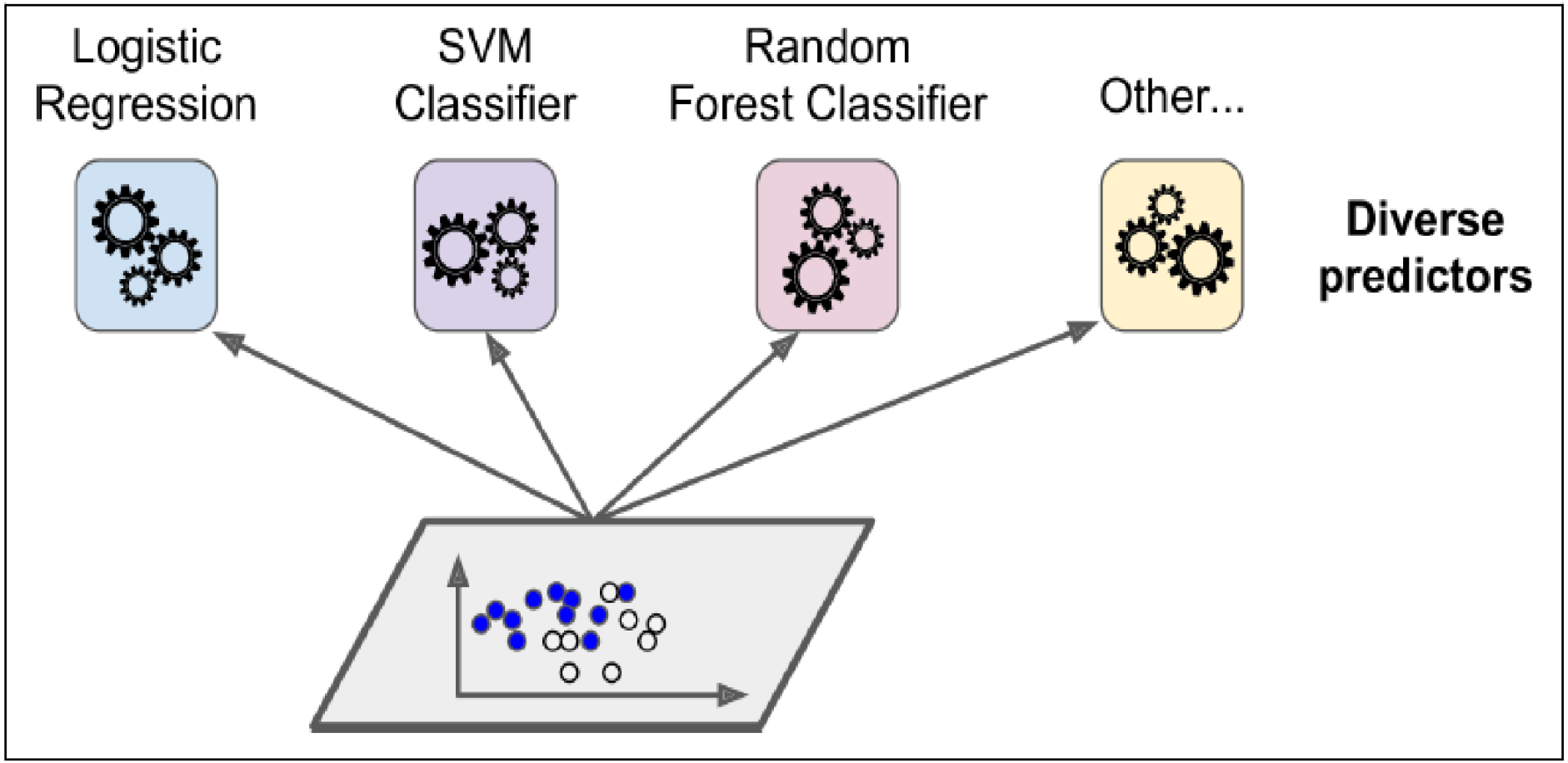
3

Stacking

Trains a meta-learner (blender) that takes the predictors' predictions as input

A voting classifier trains different predictors on the same data

Training multiple classifiers — logistic regression, SVM, random forest and other diverse predictors

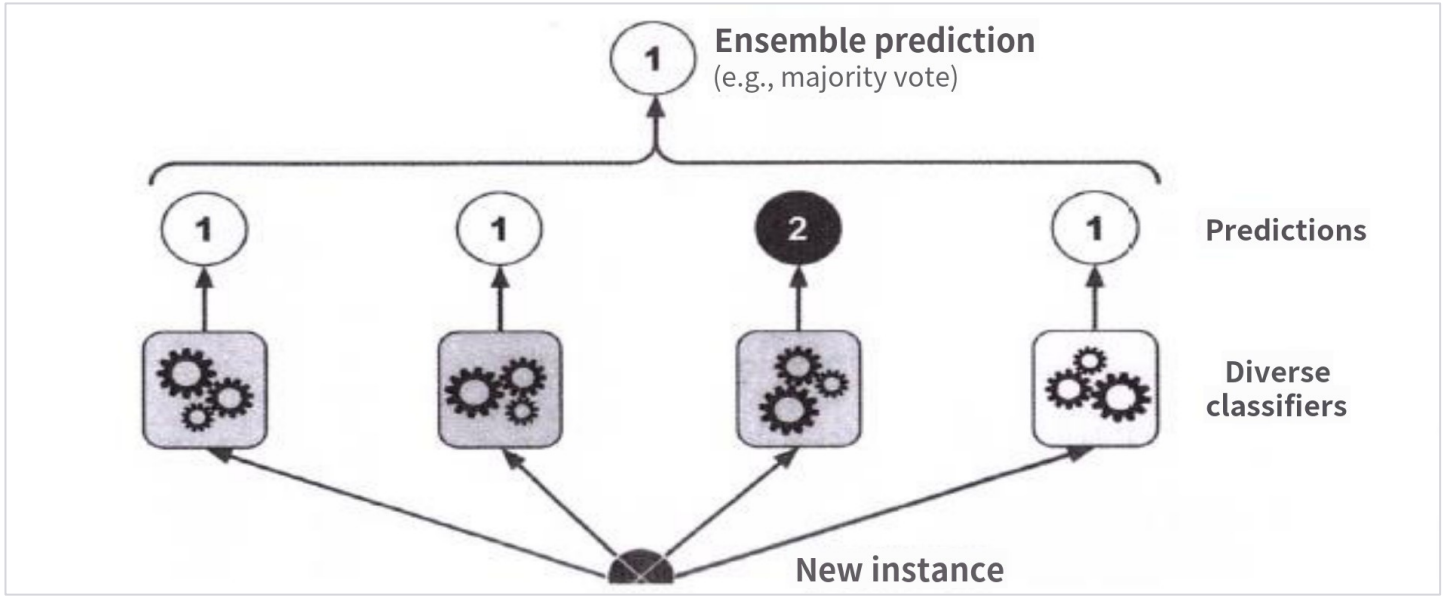


Enough diverse weak learners become a strong learner

Hard voting classifier predictions — picks the class of a new instance by majority vote

Voting Ensemble

- Each model votes and the majority picks the class (figure: 1·1·2·1 → 1)
- VotingClassifier is in sklearn's ensemble package
- The practice uses the Adult dataset



With a weak model mixed in, the ensemble trails a single model

Voting ensemble (voting='soft') — logistic regression + random forest, Adult dataset

[Input] Prompt · Claude Code (Code tab)

```
Test a voting ensemble on the Adult dataset.
- voting='soft'
- Logistic regression (random_state=2019) + random forest
(n_estimators=100, max_depth=10, random_state=2019)
- Baselines: a single random forest with the same settings,
and logistic regression alone
- Hold out 20% as test (random_state=2019) and print all
three accuracies.
```

What to check

- Did all models use the same test set?
- What is the logistic regression accuracy alone?
- Do the two random forests use the same settings?

```
Voting Ensemble Acc : 82.51%

Single Random Forest Acc : 83.56%
```

Results

- Voting ensemble 82.51% (RF: 100 trees, depth 10)
- Single random forest 83.56% (50 trees, depth 5: different settings)
- Same-setting rerun (20% test, seed 2019): ensemble 82.21%, single RF 85.48%, logistic regression 78.30%. In this setup the ensemble trails the single RF

Average blending merges the predictions of n models by averaging

Average blending — the most widely used technique on Kaggle

Regression

- Divide the models' predicted values by n and sum them

Classification

- Divide each class's probability by n and sum them
- Pick the class with the highest resulting probability

Average blending averages per-fold predicted probabilities

Average blending — 5-fold StratifiedKFold random forest probabilities, divided by skf.n_splits and summed

[Input] Prompt · Claude Code (Code tab)

```
Implement average blending on the Adult dataset.
- Model: random forest (n_estimators=50, max_depth=5,
random_state=2019)
- Split x_train with StratifiedKFold(n_splits=5) and train in
each fold.
- Print each fold's training and validation accuracy, then the
mean as the Cross Validation Score.
- Build y_pred by adding x_test predicted probabilities divided
by skf.n_splits.
- Compare test accuracy with a single model on the same x_test.
```

What to check

- Is x_test kept out of the training folds?
- Were probabilities divided by skf.n_splits before adding?
- Are test scores compared with test scores, not CV scores?

```
0 Fold, train Accuracy : 83.37%, validation Accuracy : 83.79%
1 Fold, train Accuracy : 84.00%, validation Accuracy : 83.24%
2 Fold, train Accuracy : 83.95%, validation Accuracy : 83.70%
3 Fold, train Accuracy : 83.87%, validation Accuracy : 83.79%
4 Fold, train Accuracy : 83.96%, validation Accuracy : 83.58%
Cross Validation Score : 83.62%

Single Random Forest Acc : 83.56%
```

Results

- Per-fold validation accuracy 83.24–83.79%
- Cross-validation score 83.62%
- 83.62% is a CV score and 83.56% a test score, so they are not compared directly. Same test set rerun: blending 84.51%, single RF 84.56%, about equal

04

PART 4 · Stacking

Stacking

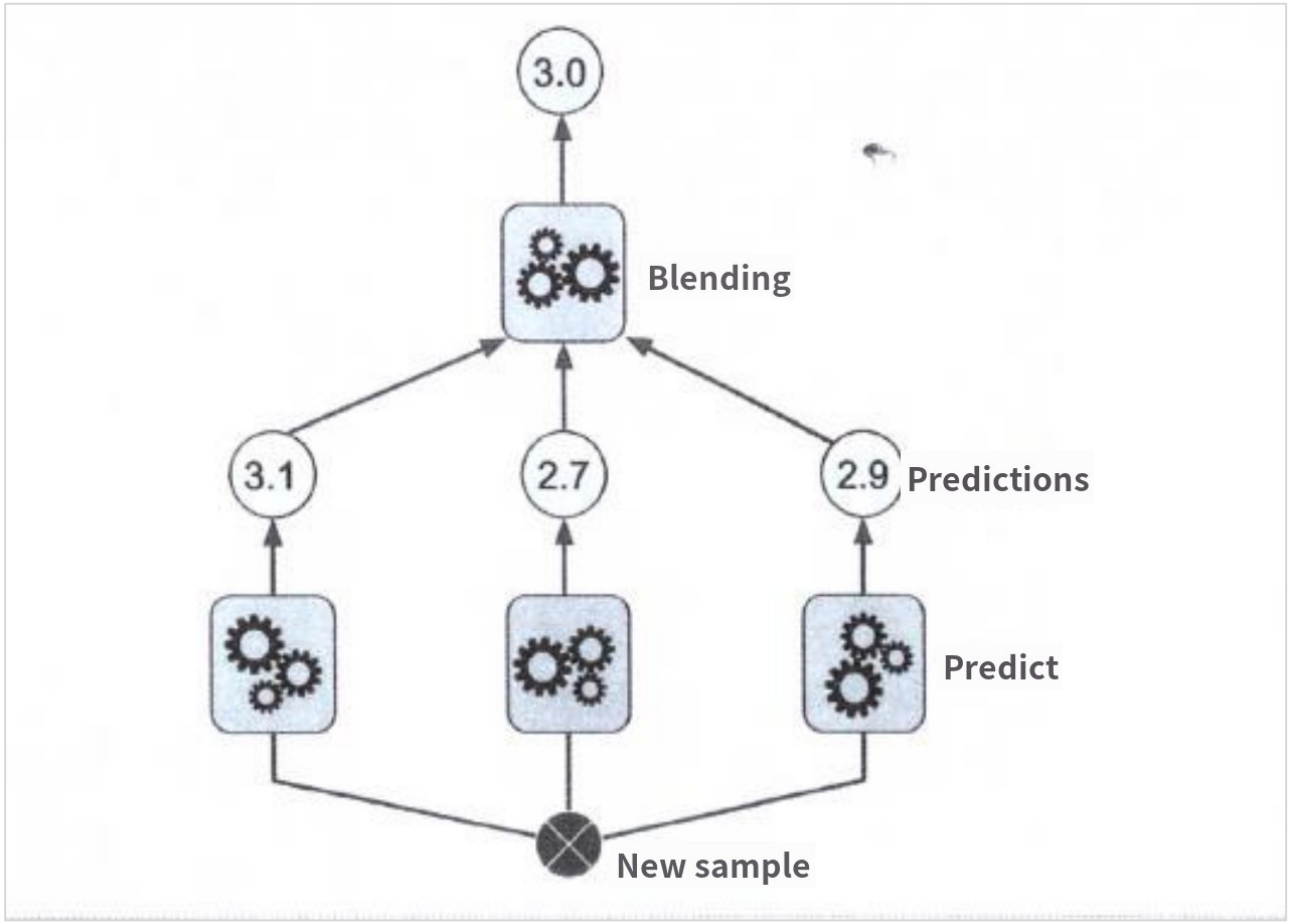
Meta-models that aggregate predictors' predictions, and CV-based stacking

Stacking trains a separate model to combine predictions

Short for Stacked Generalization — the final predictor is called a blender or meta-learner

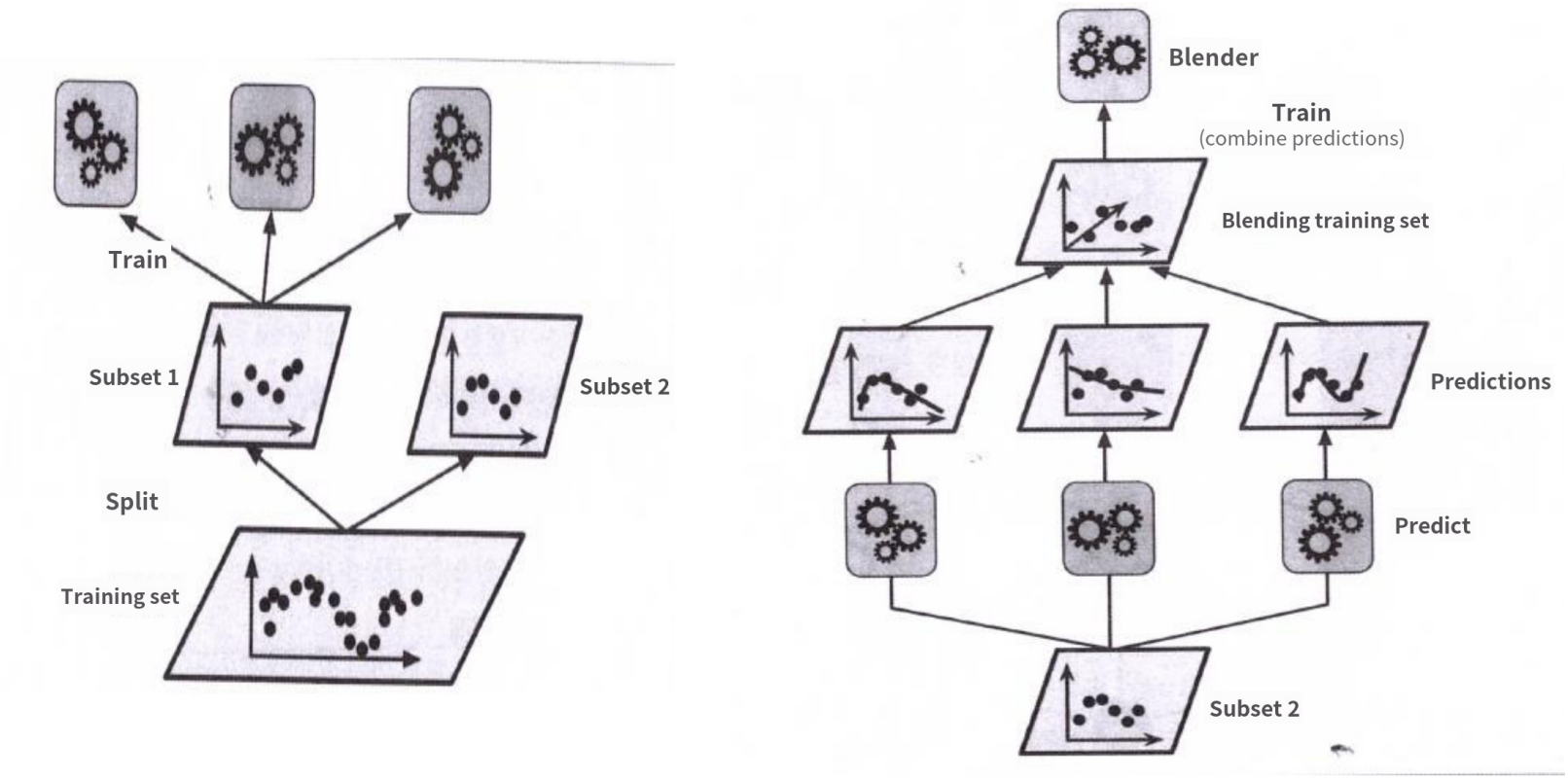
Reading the figure

- Three predictors predict 3.1, 2.7 and 2.9 for a new sample
- The blender takes these three predictions as input and outputs a final prediction of 3.0
- Instead of combining by a fixed rule as voting and averaging do, it learns how to combine



The blender trains on a hold-out set the first layer never saw

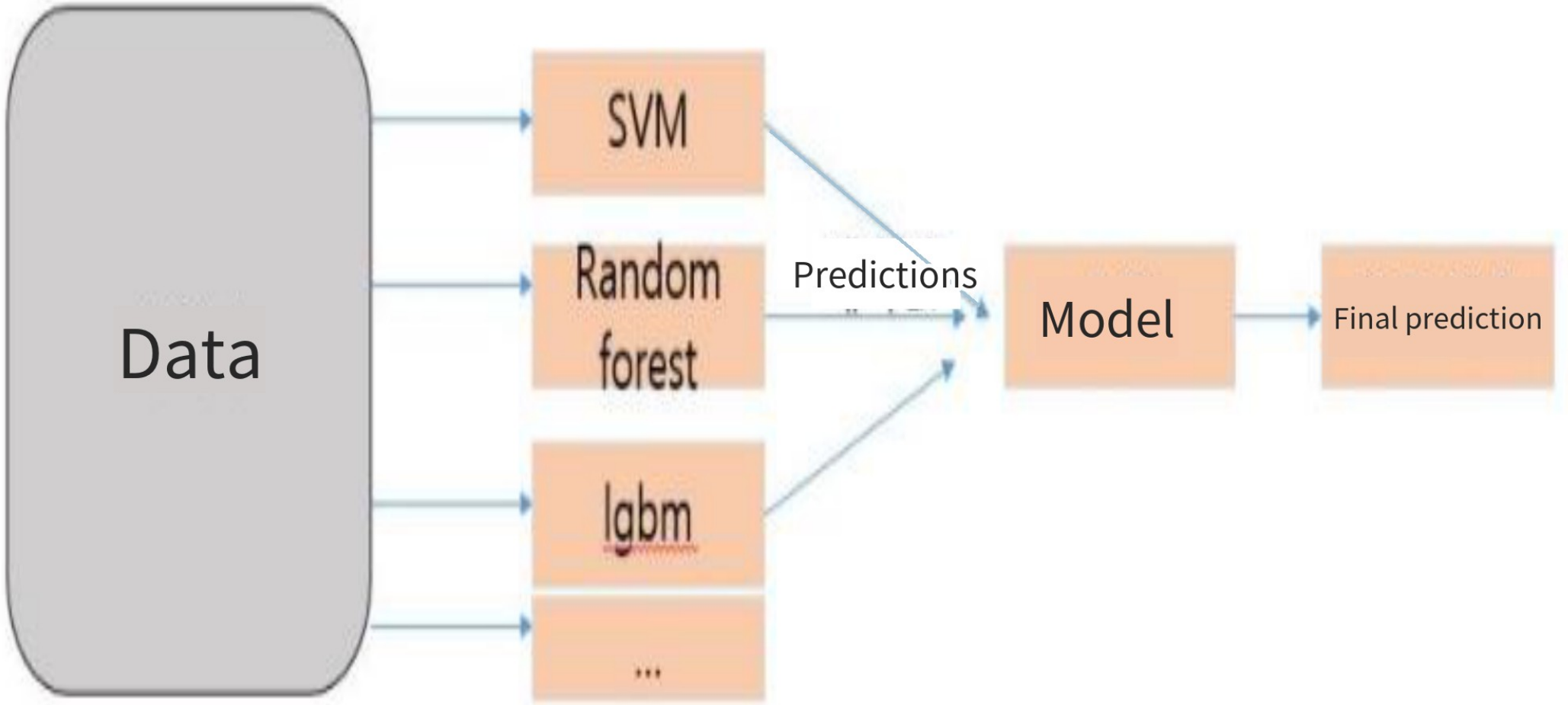
Stacking training process — left: training the first layer, right: training the blender



Train the first layer on subset 1 (training set) and the blender on predictions for subset 2 (hold-out set)

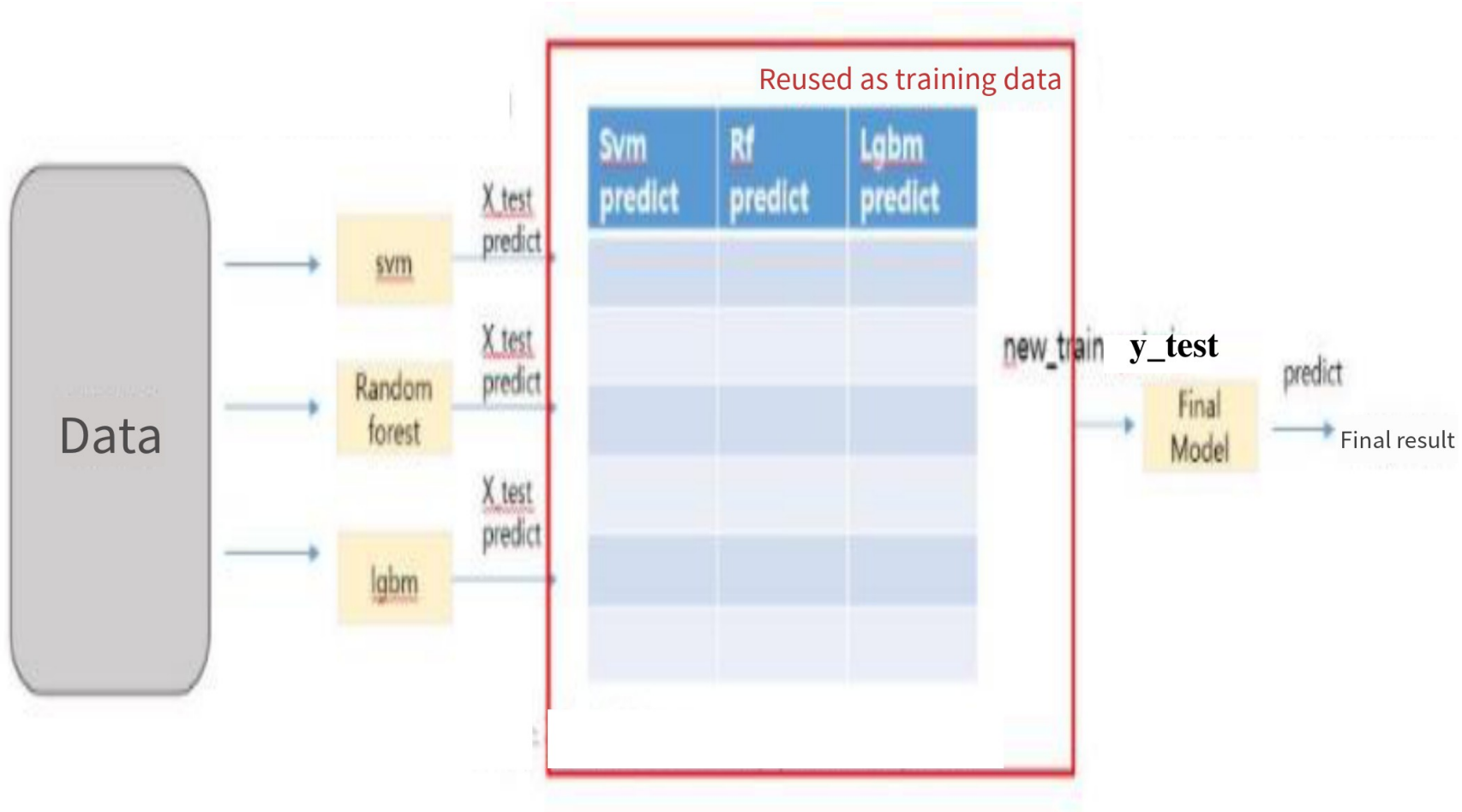
Several models' predictions become the final model's inputs

Basic stacking ensemble structure — SVM, random forest, LightGBM, etc.



Basic stacking trains stage two on test-set predictions

Basic structure (detail) — stack X_test predictions as columns into new_train, then train the final model with y_test

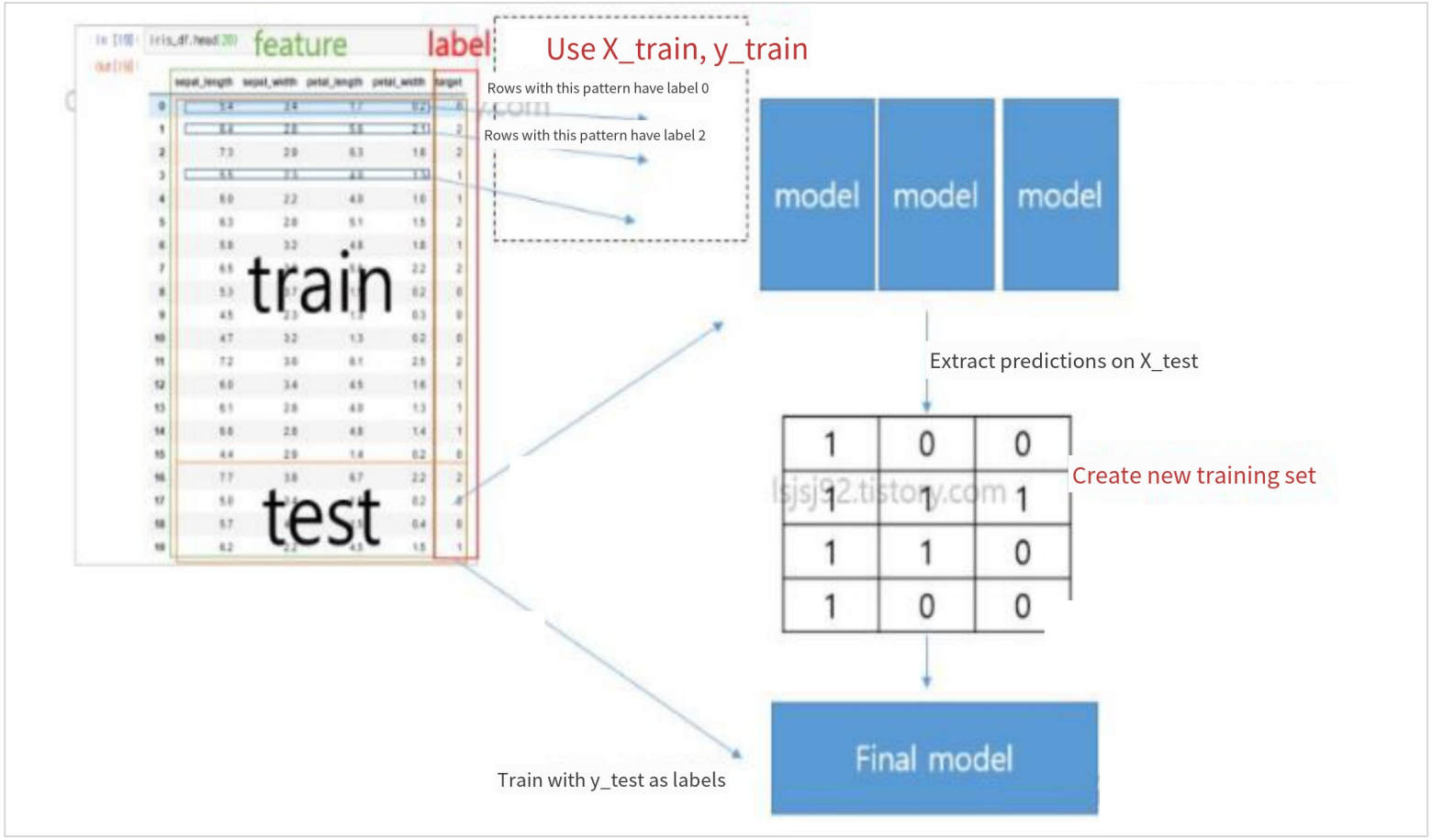


The final model learns a training set of base-model predictions

Final structure — train base models → create a new training set from X_test predictions → train with y_test as labels

Caution

- In this approach the final model learns from y_test
- Then the test set can no longer measure performance fairly
- CV-based stacking, next, solves this problem

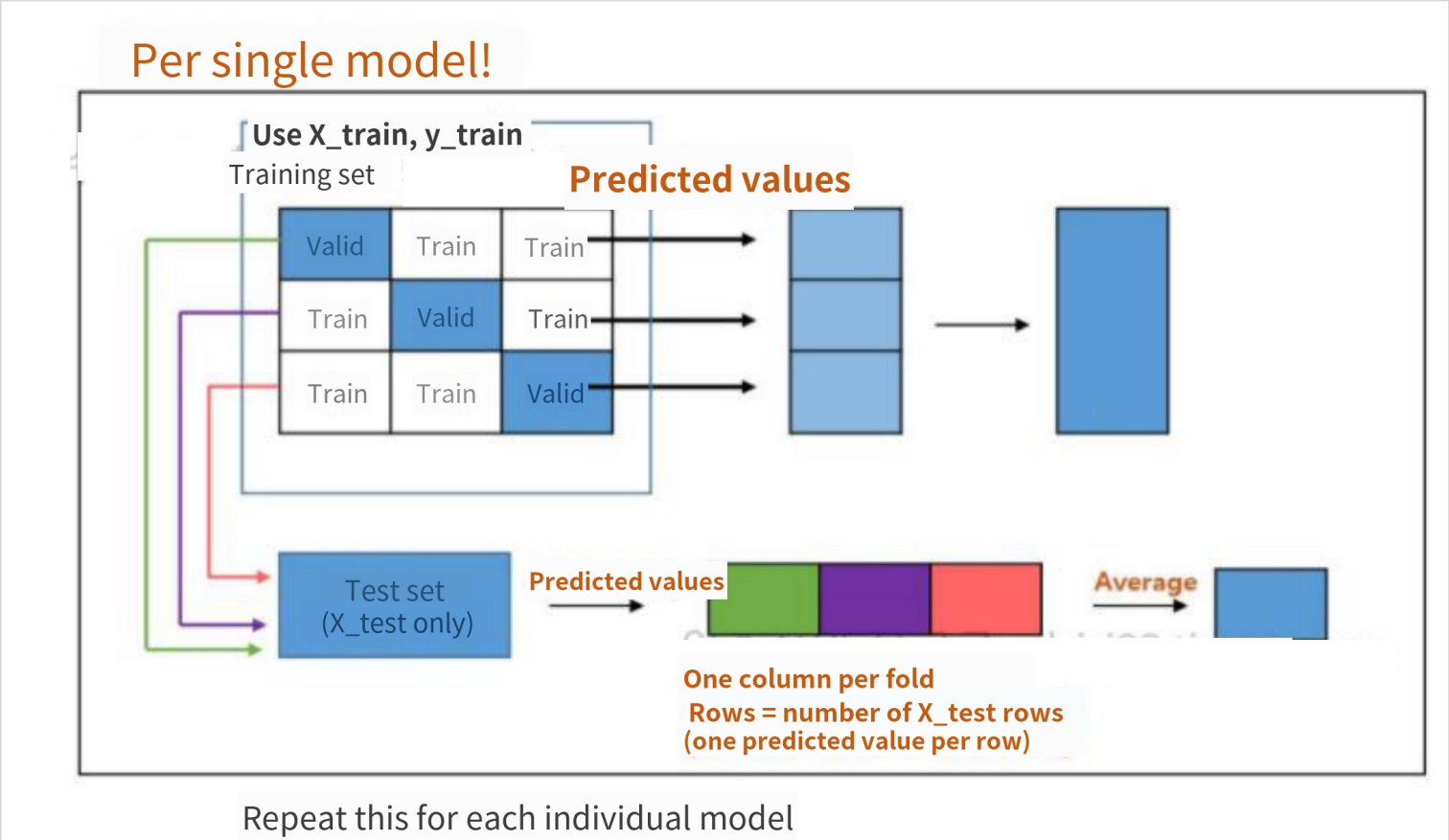


CV-based stacking predicts validation and test in every fold

[Note] Run 3 folds with one model, and repeat this for each individual model

Per single model

- Predicting the validation fold in each fold and collecting them yields one column of predictions for all of X_train
- X_test is also predicted in each fold. This yields one column per fold, with as many rows as X_test
- Averaging those columns gives one column of test predictions

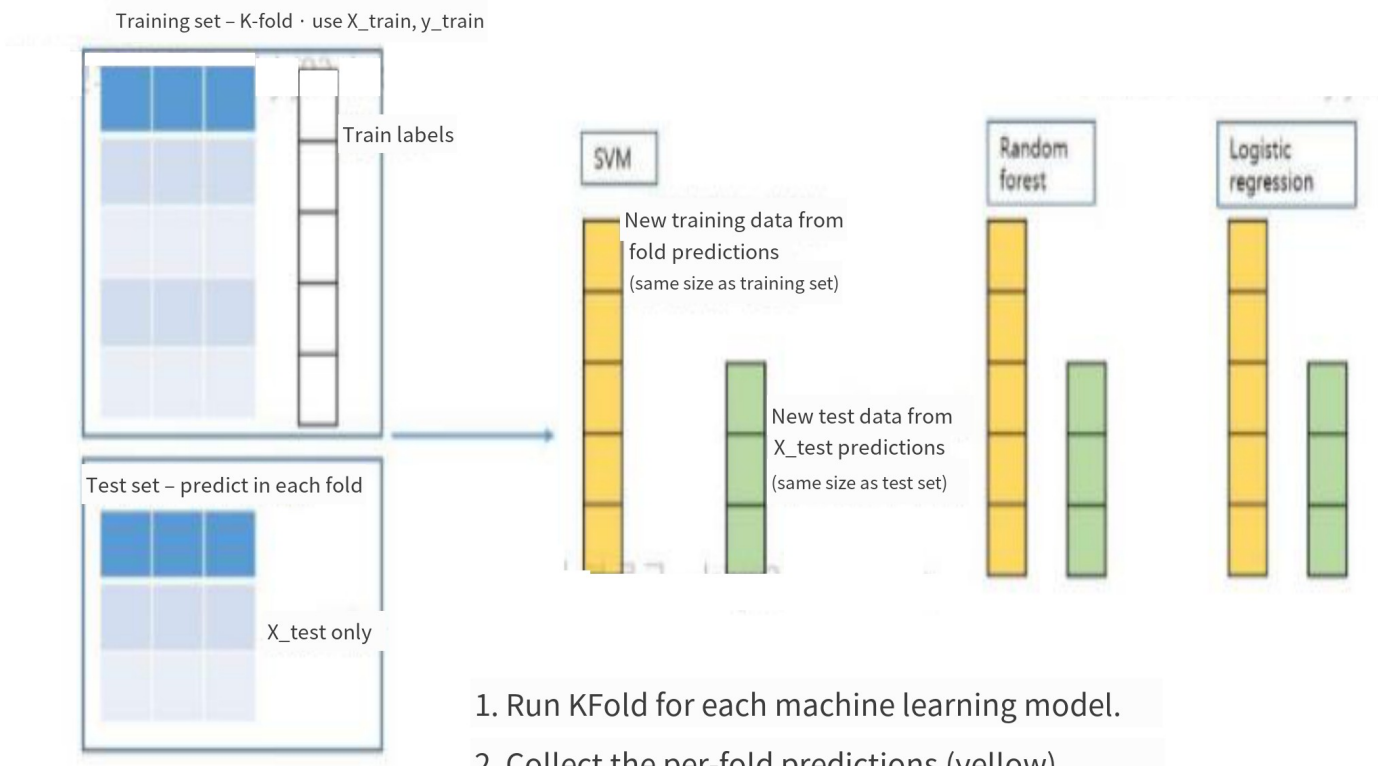


CV predictions match the training set, pairing with y_train

K-fold run per model — yellow: per-fold validation predictions (new training data), green: average of X_test predictions

Procedure

- Run KFold for each machine learning model
- Collect the per-fold predictions (yellow)
- Average and collect the per-fold predictions on the original X_test (green)

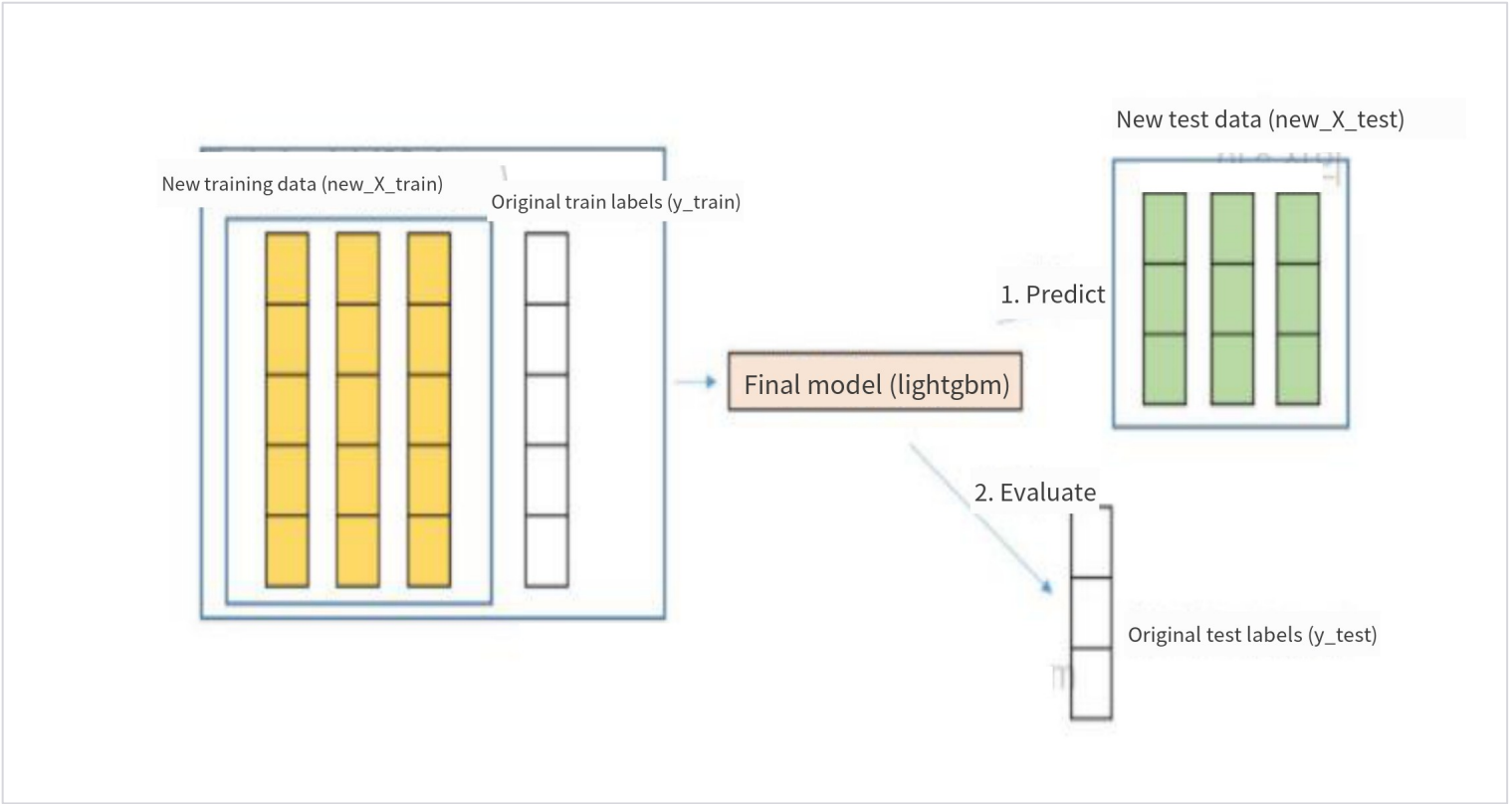


The final model trains on y_train and is evaluated on y_test

Stacking final model — new training data (new_X_train) + original training labels (y_train) → final model (LightGBM)

Final model

- Collect fold validation predictions into new_X_train
- Collect X_test predictions into new_X_test
- LightGBM trained on new_X_train and y_train predicts the final y_test from new_X_test



Fair validation is the starting point for tuning and ensembles

What the four parts established

1

Cross-Validation

Split classification problems with stratified K-fold. On sorted data, KFold gave 91.33% and stratified 96.67%

2

Hyperparameter Tuning

Grid → random → Bayesian saves trials step by step. The acquisition function weighs exploitation and exploration together

3

Ensembles and stacking

Mixing in a low-performing model can leave the ensemble below a single model. Stacking's meta-model learns from CV predictions to prevent leakage